Automata and Formal Languages

Dr. Sankhadeep Chatterjee

Course Formalities

- Contact – 3L + 1T
- Credit – 4
- Evaluation
  - Relative Grading
  - Quiz – 20 (Best of Two)
  - Mid Sem – 30
  - End Sem – 50 (Scaled down from 100)
- Announcements, Slides, Course materials will be made available in Course Google Classroom (Link will be shared)

References

- “Introduction to Automata Theory, Languages, and Computation” Ullman, Motwani, Hopcroft (Pearson) (**Main Text**)
- “Automata and Computability” Dexter Kozen (Springer)
- “Introduction to the Theory of Computation” Michael Sipser (Cengage)
- “Introduction to Languages and the Theory of Computation” John C Martin (Tata Mcgraw-Hill)
- “An Introduction to Formal languages & Automata” Peter Linz (Jones & Bartlett)
- “Elements of the Theory of Computation”, H. R. Lewis and C. H. Papadimitriou. (Pearson)

Syllabus

- **INTRODUCTION**
  - Basic Mathematical Objects: Sets, Logic, Functions, Relations, Strings, Alphabets, Languages; Mathematical Induction: Inductive Proofs, Principles, Recursive Definitions, Set Notation.

- **FINITE AUTOMATA AND REGULAR EXPRESSIONS**
  - Finite State Systems, Deterministic Finite Automata; Nondeterministic Finite Automata, Nondeterministic Finite Automata with Epsilon, Applications, Kleene' Theorem; Two-way Finite Automata, Finite Automata with Output, Regular Languages & Regular Expressions, Properties of Regular Sets: The Pumping Lemma for Regular Sets, Closure Properties, Decision Properties of Regular Languages, Equivalence and Minimization of Automata, Moore and Mealy Machines.

- **CONTEXT FREE GRAMMARS**
  - Definition, Derivation Trees & Ambiguity, Inherent Ambiguity, Parse Tree, Application of CFG, Simplification of CFG, Normal Form of CFG, Chomsky Normal Form and Chomsky Hierarchy, Unrestricted Grammars, Context-Sensitive Languages, Relations between Classes of Languages, Properties of Context Free Languages: The Pumping Lemma, Closure Properties, Decision Properties of CFL.

Syllabus

- **PUSHDOWN AUTOMATA**
  - Definitions, Languages of PDA, Equivalence of PDA and CFG , Deterministic PDA.

- **TURING MACHINES**
  - Turing Machine Model, Language of a Turing Machine (TM), Programming Techniques of the TM, Variations of TM, Multiple TM, One-Tape and Multi-Tape TM, Deterministic and Non deterministic TM, Universal TM, Church Thesis, Recursively Enumerable Languages, Decidability, Reducibility, Intractable Problem Classes of Problems NP Hard, NP Complete.

Introduction

- Why study Automata and Formal Languages (a.k.a. Theory of Computations)?
  - What a computing machine can do and what it can not do?
  - Which problems can be solved efficiently by computer?
  - Which problems are hard to solve (intractable)?

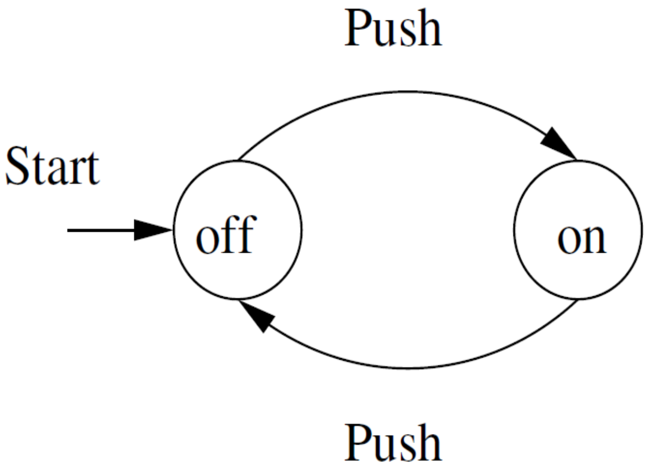
Finite Automata

- Dictionary Definition of an Automaton
  - noun (plural automata)
  - A moving mechanical device made in imitation of a human being.
  - A machine that performs a function according to a predetermined set of coded instructions.
- Automata (Greek) – *Self Acting*
- Mathematical model of a computing device / software

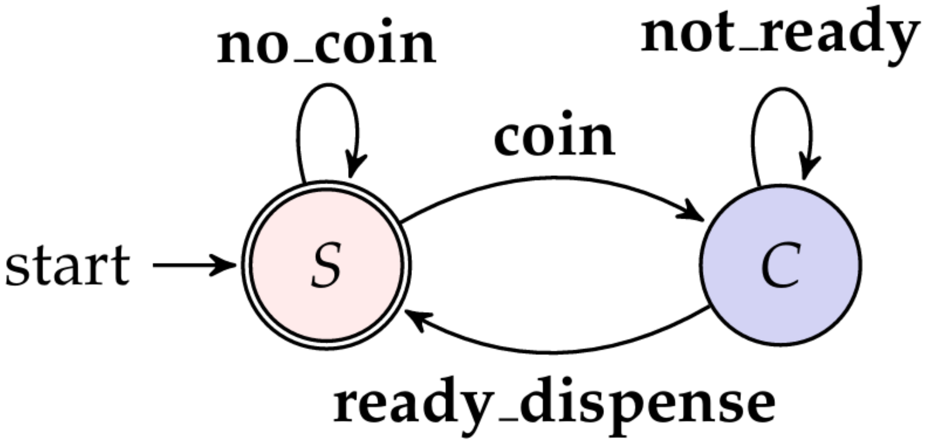
Example

- Switch as an Finite Automaton

- The device remembers whether it is in the “on” state or the “off” state

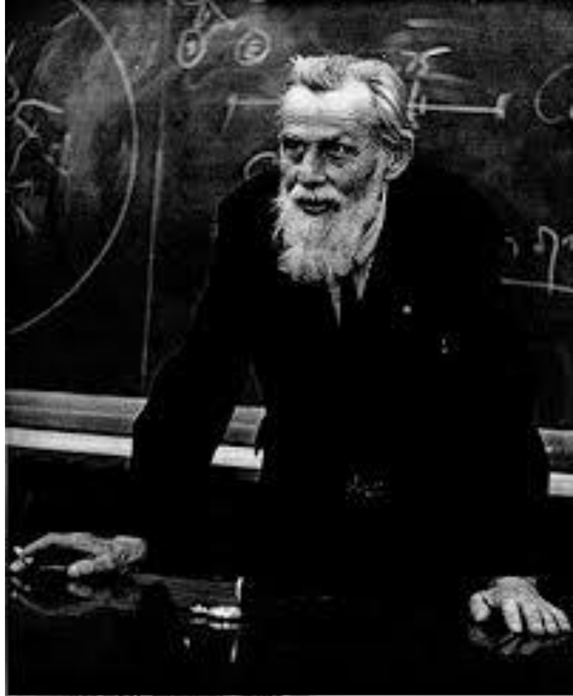


- Vending Machine as Finite Automata



Finite Automata

- Introduced first by two neuro-psychologist Warren S. McCullough and Walter Pitts in 1943 as a model for human brain!
- Finite automata can naturally model microprocessors and even software programs working on variables with bounded domain
- Capture so-called regular sets of sequences that occur in many different fields (logic, algebra, regEx)
- Nice theoretical properties
- Applications in digital circuit/protocol verification, compilers, pattern recognition, etc.



Turing Machine

- Introduced by Alan Turing as a simple model capable of expressing any imaginable computation
- Turing machines are widely accepted as a synonyms for algorithmic computability (Church-Turing thesis)
- Using these conceptual machines Turing showed that first-order logic validity problem (one of the most famous problem of 20th century posed by David Hilbert) is non-computable,
- i.e. there exists some problems for which you can never write a program no matter how hard you try!

Central Concepts of Automata

- **Alphabets:** An alphabet is a finite, nonempty set of symbols. Conventionally, we use the symbol Σ for an alphabet.
  1. $\Sigma = \{0, 1\}$ is a binary alphabet
  2. $\Sigma = \{a, b, \ldots, z\}$ set of all lower case letters
  3. The set of all ASCII characters, or the set of all printable ASCII characters

- **Strings:** A string (or sometimes word) is a finite sequence of symbols chosen from some alphabet.
  1. For example "01101" is a string from the binary alphabet $\Sigma = \{0, 1\}$.
  2. The string "111" is another string chosen from this alphabet.

Central Concepts of Automata

- **Empty String:** The empty string is the string with zero occurrences of symbols. Usually denoted by $\boldsymbol{\epsilon}$

- **Length:** The number of positions for symbols in the string.
  - The standard notation for the length of a string w is $|w|$
  - For example, $|011| = 3$ and $|\epsilon| = 0$

- **Powers of an Alphabet:** Σ^k to be the set of strings of length k, each of whose symbols is in Σ

  Considering $\Sigma = \{0, 1\}$,

  $\Sigma^0 = \{\epsilon\}$, $\Sigma^1 = \{0, 1\}$, $\Sigma^2 = \{00, 01, 10, 11\}$

  $\Sigma^3 = \{000, 001, 010, 011, 100, 101, 110, 111\}$

Central Concepts of Automata

- The set of all strings over an alphabet Σ is conventionally denoted Σ^*
  - $\{0,1\}^* = \{\epsilon, 0, 1, 00, 01, 10, 11, 000, 001, 010, 011, 100, 101, 110, 111, \dots\}$

- In other words

$$\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \cdots$$

- The non empty set of strings are denoted as follows

$$\Sigma^+ = \Sigma^1 \cup \Sigma^2 \cup \cdots$$

$$\Sigma^* = \Sigma^+ \cup \{\epsilon\}$$

Central Concepts of Automata

- **Language:** A set of strings all of which are chosen from some Σ^*, where Σ is a particular alphabet, is called a language.
  - If Σ is an alphabet, and $L \subseteq \Sigma^*$ then L is a ***Language over Σ***
  - The language of all strings consisting of n 0's followed by n 1's, for some $n \geq 0$: $\{\epsilon,\ 01, 0011, 000111, \ldots\}$
  - The set of strings of 0's and 1's with an equal number of each: $\{\epsilon, 01, 10, 0011, 1010, 0101, \ldots\}$
  - Ø denotes **Empty Language**
  - $\{\epsilon\}$ is a language consisting of only empty string (Note: $\emptyset \neq \{\epsilon\}$)

Central Concepts of Automata

- **Problems:** A problem is the question of deciding whether a given string is a member of some particular language

- Given a string w in Σ^* decide whether or not w is in L